

EKS 기반 GitOps 배포 플랫폼 구축

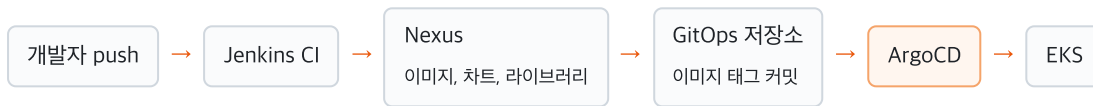
역할 QA 신규 플랫폼 설계와 구축, 파이프라인 표준화, 운영

환경 EKS (Karpenter, Fargate), Terraform, Jenkins, Nexus, ArgoCD, Helm, External Secrets, Gateway API

배경

- Jenkins가 산출물을 SSH로 서버에 보내고 원격 스크립트를 실행하는 Push 방식 배포
- 서비스마다 파이프라인 스크립트가 달라 빌드와 배포 기준이 한 곳에 없는 상태
- 어떤 버전이 어디에 배포됐는지 Git으로 추적 불가
- 설정 파일에 DB 비밀번호와 AWS 액세스 키가 기본값 형태로 커밋된 서비스 존재

구조



- Jenkins는 빌드, 이미지 push, GitOps 저장소 커밋까지만 담당. 클러스터 반영은 ArgoCD 전담
- Nexus는 아티팩트 저장소, Git은 "무엇을 배포할지"의 기록. 롤백은 GitOps 커밋 revert
- 저장소는 서비스 소스, CI(Dockerfile과 Shared Library), 공용 Helm 차트, GitOps, Terraform으로 분리
- 공용 차트 1개에 서비스별 values로 차이 표현. 서비스마다 차트를 만들지 않는 구조

App-of-Apps

- root Application이 환경별 apps 디렉터리를 재귀 등록. 신규 서비스 배포는 `Application.yaml` 과 `values.yaml` 2개 파일 추가로 끝
- ApplicationSet 대신 App-of-Apps 선택. 서비스마다 네임스페이스, 차트 버전, values 구조가 달라 개별 Application이 더 명확하다는 판단
- root를 애플리케이션(prune 사용)과 플랫폼용(prune 미사용)으로 분리. AppProject도 나눠 개발자와 CI 계정은 앱 범위만 접근

Jenkins Shared Library

- 원자 함수 7개: checkout, 이미지 빌드, push, 이미지 정리, GitOps values 갱신, ArgoCD sync, 알림. 함수끼리 서로 호출하지 않는 구조
- 조합 템플릿 4개: QA 자동 배포, 운영 빌드, 운영 배포(수동 승인), 수동 검증용
- 새 요구는 조합 템플릿에서 처리하고 원자 함수는 유지. 앱 서비스 8개와 공용 라이브러리 16종에 적용
- 운영은 Build와 Deploy Job 분리. Deploy가 실행 시점에 Nexus의 최신 태그를 직접 조회하고 승인 게이트를 거치는 구조
- 승인 대기기는 executor 밖에서 처리. 승인 대기 2건이 QA 빌드까지 막는 문제 예방

Terraform과 ArgoCD 관리 경계 정립

기준: 쿠버네티스 API 밖이면 Terraform, 안이면 ArgoCD. 예외는 부트스트랩뿐.

리소스	이전	이후
VPC, EKS, IAM, DB, 보안 그룹	Terraform	Terraform
ArgoCD, Karpenter	Terraform	Terraform (부트스트랩 예외)
Gateway API CRD, Traefik, ESO, ALB Controller 차트	Terraform	ArgoCD (sync wave로 순서 제어)
ALB Ingress, 내부 NLB Service	수동 <code>kubectl apply</code>	ArgoCD
ArgoCD 계정과 RBAC	Terraform patch	ArgoCD selfHeal (부분 소유)

- 계기: Traefik 모듈이 apply 시점에만 알 수 있는 값을 `for_each` 키로 써서 상류 모듈이 바뀔 때마다 plan 자체가 실패
- CRD의 `last-applied-configuration` 어노테이션이 한계의 95%(248KB)까지 찬 상태
- 이관 순서: Application 추가(자동 sync 없이) → diff로 동일 확인 → `terraform state rm` → 수동 sync로 인수 → 기능 검증. destroy 미사용
- 결과: `terraform plan` No changes, Application 16개 Synced/Healthy, 어노테이션 248KB → 0, 서비스 중단 없음

시크릿과 설정값 분리 표준 수립

값	관리 위치
시크릿	Secrets Manager + External Secrets (서비스별 시크릿 1개)
비시크릿 설정값	GitOps values의 ConfigMap
AWS 자격 증명	사용하지 않음. EKS Pod Identity
프론트엔드 빌드 변수	소스 저장소의 프로필별 예시 파일 (빌드 시 이미지에 포함)

- 설정 파일의 평문 기본값 제거 6개 서비스. 정적 액세스 키 문자열 0건 확인
- IRSA 대신 Pod Identity 선택. 공용 라이브러리에 STS 모듈이 없어 IRSA 토큰 단계를 예러 없이 건너뛰고 노드 역할로 넘어가는 동작 확인. Pod Identity는 라이브러리 수정 없이 적용 가능
- IAM은 서비스 전체 권한 정책 대신 서비스별 역할과 실제 호출 액션만 허용

제어면 Fargate 분리

- ArgoCD, Karpenter, CoreDNS, ALB Controller를 Fargate로 이동. 워크로드 노드가 모두 사라져도 배포 도구가 살아 있는 구조
- 계획을 뒤집은 순서: 노드 CPU requests가 이미 85~90%라 requests부터 올리면 Karpenter가 노드를 추가로 띄우는 상황. Fargate 프로파일을 먼저 만들어 새 파드가 처음부터 Fargate에 뜨도록 변경
- ALB Controller는 replica 2로 늘린 뒤 이동. webhook 엔드포인트가 0이 되면 클러스터 전체 Ingress 변경이 막히는 구조라서
- 결과: 앱 파드 재시작 0건, 노드 교체 0건, ArgoCD UI 중단 없음
- 전환 근거는 안정성이 아니라 제어면과 데이터플레인 분리로 기록. Fargate는 OS 패치 재시작 시점을 AWS가 정한다는 점도 함께 명시

운영 중 문제와 해결

문제	원인	해결
이미지 태그 덮어쓰기	Job 재생성 시 빌드 번호가 1로 리셋	환경-날짜-빌드번호-커밋SHA8 태그와 OCI 라벨. 운영 레지스트리는 삭제 권한 없음
Nexus 디스크 100%로 push 실패	"최근 N개 보존" 기능 미지원, 업로드 임시 blob 누적, Compact 작업 미등록	4단 삭제 체인 규명, 파이프라인 끝에 보존 정리(안전장치 6중), Nexus 스케줄로 회수
동시 빌드 실패	서비스 간 Gradle 캐시 공유로 lock 경합, GitOps 동시 push 충돌	서비스별 캐시 격리, fetch 후 재적용 재시도 (최대 3회, 이후 실패 처리)
Helm upgrade 후 ArgoCD 권한 소실	차트 기본값의 빈 <code>policy.csv</code> 가 기존 정책을 덮어씀. 관리자는 RBAC를 우회해 발견이 늦음	GitOps로 이관 후 일부러 망가뜨려 25초 내 자동 복구 확인. AppProject로 권한 분리
Poll SCM 무한 재빌드	파이프라인이 체크아웃한 GitOps 저장소가 폴링 대상에 자동 포함되고, 매 실행 커밋이 다시 빌드를 유발	보조 저장소는 <code>git clone</code> 으로 받아 앱 소스만 폴링
새 공용 라이브러리 미반영	Gradle이 SNAPSHOT 갱신을 24시간에 1회만 확인	확인 주기 0초로 변경, 빌드 캐시는 유지
파드 간 호출 타임아웃 (8~10초)	클러스터 내부에서 공개 도메인 호출 시 LB 헤어핀 트래픽이 버려짐	서버 간 호출은 내부 주소, 브라우저 진입점만 공개 도메인

결과

항목	결과
신규 서비스 배포	설정 파일 2개 추가
파이프라인	원자 함수 7개, 조합 템플릿 4개로 서비스 8개와 공용 라이브러리 16종 통일
Terraform 모듈	커스텀 모듈 15종
Nexus 디스크	70% → 29% (약 20GB 회수)
프론트엔드 이미지	1.09GB → 292MB (Multi-stage 빌드, standalone 출력)
태그·레지스트리 전환	서비스 8개, 파드 16개 재시작 0건

남은 작업

- 운영 환경 클러스터와 IaC 구성 (진행 중)
- GitOps 저장소 직접 push로 승인 게이트를 우회할 수 있는 경로 차단 (보호 브랜치 설계 필요)
- 노드 정의(NodePool)의 ArgoCD 이관 검토